

TECHNICAL REFERENCE MANUAL

ANITTA

Voice agent for insurance offers

Conversational agent · Voice · TrustCore integration · Offer automation · Configuration · Security · Roadmap

Document	Version	Date	Basis	Classification
AN-TRM-001	1.0	4 November 2026	anitta-main · v0.7.2+15	Confidential

Document control

PROPERTY	VALUE
Title	Anitta — Technical Reference Manual
Identifier	AN-TRM-001
Version	1.0 — 4 November 2026
Status	Released
Classification	Confidential. Contains security findings, including references to exposed credentials; distribute to the technical owner and authorised contractors only.
Companion document	TC-TRM-001 TrustCore — Technical Reference Manual (3 November 2026)

Basis of this document

ARTEFACT	VERSION ANALYSED
Mobile application	`anitta-main/`, Flutter (FlutterFlow) project, pubspec version `0.7.2+15`, bundle `ch.neotralo.anitta`
Firebase	Project `anitta-e82d3`: rules, functions scaffold and client configuration from the repository
TrustCore	Back end, database and automation package as analysed in TC-TRM-001
Not included	Deployed Firestore configuration and rules, system prompt text, AI proxy implementation, Power Automate desktop script

Audience and use

READER	RECOMMENDED PATH
Product owner / CTO	Chapters 1, 2, 13 and 16
Mobile developer	Parts II and III, chapter 15, appendices A and B
AI / prompt engineer	Chapters 5, 11 and 16
TrustCore developer	Chapters 7, 12 and appendix C
Security / compliance	Chapters 12 and 13

Conventions

Identifiers from code are set in `monospace`. “TC n.m” refers to section n.m of TC-TRM-001. Behaviour of components outside the repository is described from the interfaces the app uses and is marked as such. Secret values found in the code are referred to by location only and are never reproduced.

Contents

PART ONE · PRODUCT AND CONTEXT

1 Executive summary 6
1.1 Anitta at a glance 6
1.2 What Anitta does 6
1.3 Architecture in one paragraph 7
1.4 Headline findings 7
1.5 Sources and method 7

2 Product and user journeys 8
2.1 Journey A — voice conversation 8
2.2 Journey B — typed conversation 8
2.3 Journey C — ordering an offer 8
2.4 Journey D — demo automation 9
2.5 Supporting screens 9

3 System context 10
3.1 External services 10
3.2 Relationship to TrustCore 11

4 Technology stack 12

PART TWO · HOW ANITTA WORKS

5 Conversational agent 15
5.1 Turn sequence 15
5.2 Prompt assembly 16
5.3 Model parameters 16
5.4 What the agent can and cannot do today 17

6 Voice input and output 18
6.1 Speech recognition 18
6.2 Speech synthesis 18

7 Ordering offers through TrustCore 19
7.1 End-to-end sequence 19
7.2 Data loaded at launch 19
7.3 Offer creation request 20
7.4 What happens after the 201 20
7.5 Demo automation 20

8 Conversation memory and state 21
8.1 Two conversation stores 21
8.2 Device state 21

PART THREE · APPLICATION AND INTERFACES

9 Application architecture 23
9.1 Start-up 23

9.2 Navigation 24
9.3 Component reference 24
9.4 Localisation 24

10 Interface reference 25
10.1 Firestore operations 25
10.2 Response parsing 25

11 Runtime configuration 26

12 Identity and access 27
12.1 Consequences 27
12.2 Target model 27

PART FOUR · SECURITY, OPERATIONS AND ROADMAP

13 Security assessment 29
13.1 Method 29
13.2 Risk heat map 29
13.3 Summary 29
13.4 Findings 30
13.5 Data protection 33

14 Code quality and technical debt 34
14.1 Strengths 34
14.2 Debt register 34

15 Build and release 35
15.1 Build facts 35
15.2 Release procedure 35
15.3 Smoke test 35
15.4 Operational checks 36

16 Towards a fully autonomous agent 37
16.1 Target architecture 37
16.2 Design principles 37
16.3 Conversation flow with tools 38
16.4 Prerequisites in TrustCore 38
16.5 Roadmap 38

APPENDICES · REFERENCE DATA

A Source inventory 41

B Payload and data schemas 43

C TrustCore cross-reference 44

D Glossary 45

PART ONE

Product and Context

What Anitta is, who uses it, which journeys it supports, and the services it orchestrates on the advisor's behalf — including its dependency on the TrustCore offer platform.

1 Executive summary

2 Product and user journeys

3 System context

4 Technology stack

1 Executive summary

Anitta is an iOS assistant for insurance advisors. The advisor speaks or types; Anitta answers in natural language with a synthesised voice, knows the advisor's customers and offer templates, and places offer orders into TrustCore — which then calculates the offer on the insurer's portal and delivers the PDF to the customer without further human work.

1.1 Anitta at a glance

4
Voice languages
English, German, Swiss German, Portuguese for speech recognition

7
Cloud services
Firebase Auth, Firestore, Performance, AI proxy, OpenAI TTS, TrustCore, Power Automate

5
Screens
Home (voice), My Templates, Chat History, Language, Side menu

3
TrustCore calls
templates, customers and offer creation over the TrustCore REST API

5,633
Lines of Dart
FlutterFlow runtime plus hand-written integration and voice code

0.7.2
App version
build 15 · bundle `ch.neotralo.anitta` · iOS 14.0+

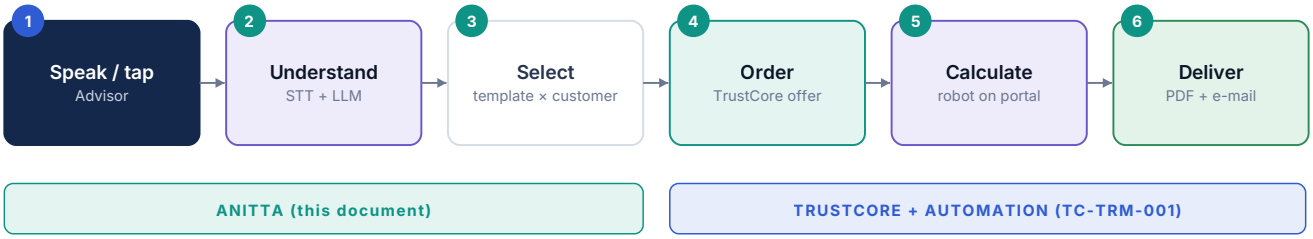
30 s
Longest utterance
dictation stops after 30 s of listening or a 3 s pause

16
Registered risks
2 critical, 5 high — see chapter 13

1.2 What Anitta does

Anitta has two capabilities that the advisor experiences as one assistant:

- **Conversation.** The advisor holds the microphone button, speaks, and receives a spoken answer. Every request carries the advisor's customer list and the available offer templates, so the language model can answer questions such as “which templates do we have for Groupe Mutuel?” or “is Müller one of my customers?” in the advisor's language. The conversation persists between sessions.
- **Offer ordering.** From *My Templates* the advisor picks a template and a customer and taps *Create Offer*. Anitta creates the offer in TrustCore, which immediately hands it to the Power Automate robot. The robot computes the offer on the insurer's portal and e-mails the PDF to the customer.



In the delivered build, steps 2 and 3 are separate: the conversation answers questions; the order is placed from the My Templates screen. Chapter 16 describes closing that gap.

Figure 1 From the advisor's request to a delivered PDF. Anitta covers the first three steps; TrustCore and its automation chain, documented in TC-TRM-001, cover the rest.

1.3 Architecture in one paragraph

Anitta is a Flutter application generated with FlutterFlow and extended with hand-written Dart (GetX controller, custom voice actions, TrustCore client). It signs in anonymously to Firebase, reads its runtime configuration — prompts, model parameters, endpoint URLs and the proxy credential — from a single Firestore document, and stores conversations per Firebase user. Language understanding runs through a Neural Swiss gateway in front of the OpenAI Responses API; speech recognition uses Apple's speech framework on the device; speech synthesis calls OpenAI's audio endpoint directly. Offers are written to TrustCore's REST API, whose existing pipeline performs the actual calculation.

1.4 Headline findings

> **DO NOT DISTRIBUTE FURTHER BUILDS BEFORE PHASE 0**
> The application embeds two secrets in its binary: an OpenAI API key used for speech synthesis, and a TrustCore bearer token used for every TrustCore call. Anyone who obtains the app package can extract both. Combined with the TrustCore authorisation weaknesses documented in TC-TRM-001, the TrustCore token gives access to customer data and offer creation. In addition, the TrustCore host compiled into the app is the legacy server `trustcode.techsolsint.com`, not production. Chapter 13 lists all sixteen findings; chapter 16 sequences remediation and the path to a fully autonomous agent.

1.5 Sources and method

This reference is based on a complete reading of the delivered repository `anitta-main` (Flutter application, iOS and Android runners, Firebase rules and functions scaffolding) and on the TrustCore analysis in TC-TRM-001. The Firestore configuration document, the system prompt text, the AI proxy implementation and the Power Automate desktop script are not part of the delivered material; where the document describes their behaviour, it is derived from how the app calls them and is marked accordingly. Secret values found in the code are referred to but never reproduced.

2 Product and user journeys

Anitta is built for one persona — the insurance advisor working on the move — and supports four journeys. Two are conversational, one places a real order, and one runs a static demonstration of the automation.

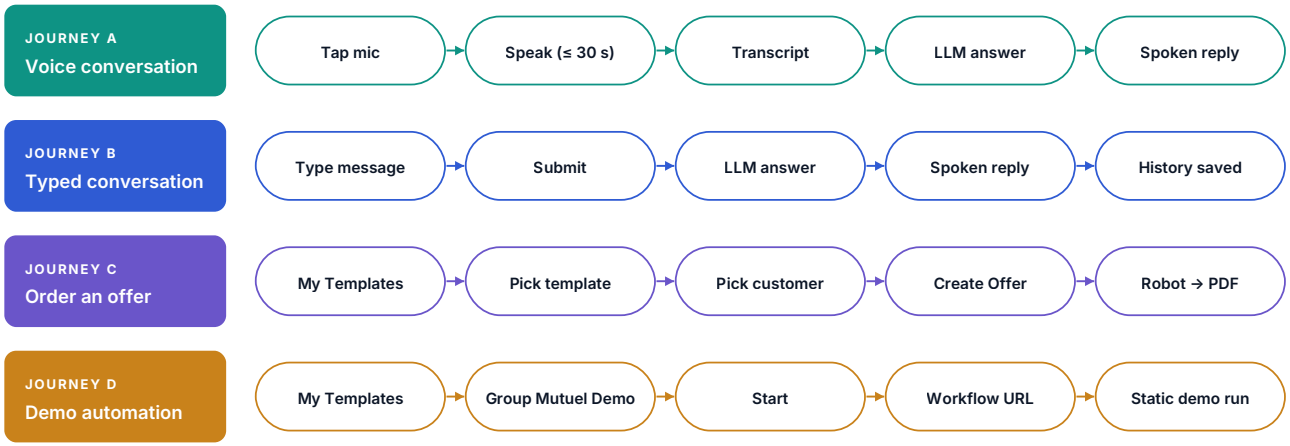


Figure 2 The four journeys supported by build 0.7.2+15.

2.1 Journey A — voice conversation

The home screen shows the Anitta logo, an animated waveform and a microphone button. Pressing it starts Apple speech recognition in dictation mode in the language chosen under *Language*; partial results stream into the transcript. Pressing stop ends recognition, sends the transcript to the language model and plays the answer aloud. The waveform animates for exactly the duration of the synthesised audio. Recognition stops automatically after 30 seconds of listening or a 3-second pause.

2.2 Journey B — typed conversation

A text field below the microphone accepts a typed message. Submitting it follows the same path as a spoken one, including the spoken reply. The text path is useful in quiet environments and for testing prompts.

2.3 Journey C — ordering an offer

1. Open the side menu and choose *My Templates*. The screen lists every template loaded from TrustCore when the app started.
2. Tap *Start* on a template card. A dialog lists the customers loaded from TrustCore.
3. Select a customer and tap *Create Offer*. Anitta posts the offer to TrustCore with the customer id, the template handle and the template name as offer title.
4. A toast confirms *Offer Created Successfully* or reports failure. From here TrustCore's automation completes the offer and e-mails the PDF to the customer.

2.4 Journey D — demo automation

The first card on *My Templates*, *Group Mutuel Demo Offer*, posts an empty request to the `workflow_url` from configuration. It starts a pre-configured automation run on static demo data and is intended for presentations; it creates no TrustCore offer.

2.5 Supporting screens

SCREEN	FUNCTION
Chat History	Read-only transcript of all turns stored for this device (“You:” / “Anitta:”), or an empty-state message.
Language	Selects the voice language: English, German, Swiss German or Portuguese. Persisted in the iOS Keychain and inserted into the system prompt.
Side menu	Home, My Templates, Chat History, Language.
Reset	App-bar action on Home that clears the current session and deletes both conversation documents in Firestore.

3 System context

Anitta is a thin, orchestrating client. Seven external services do the heavy lifting; the app's job is to combine them into one conversational experience and to hand orders to TrustCore.

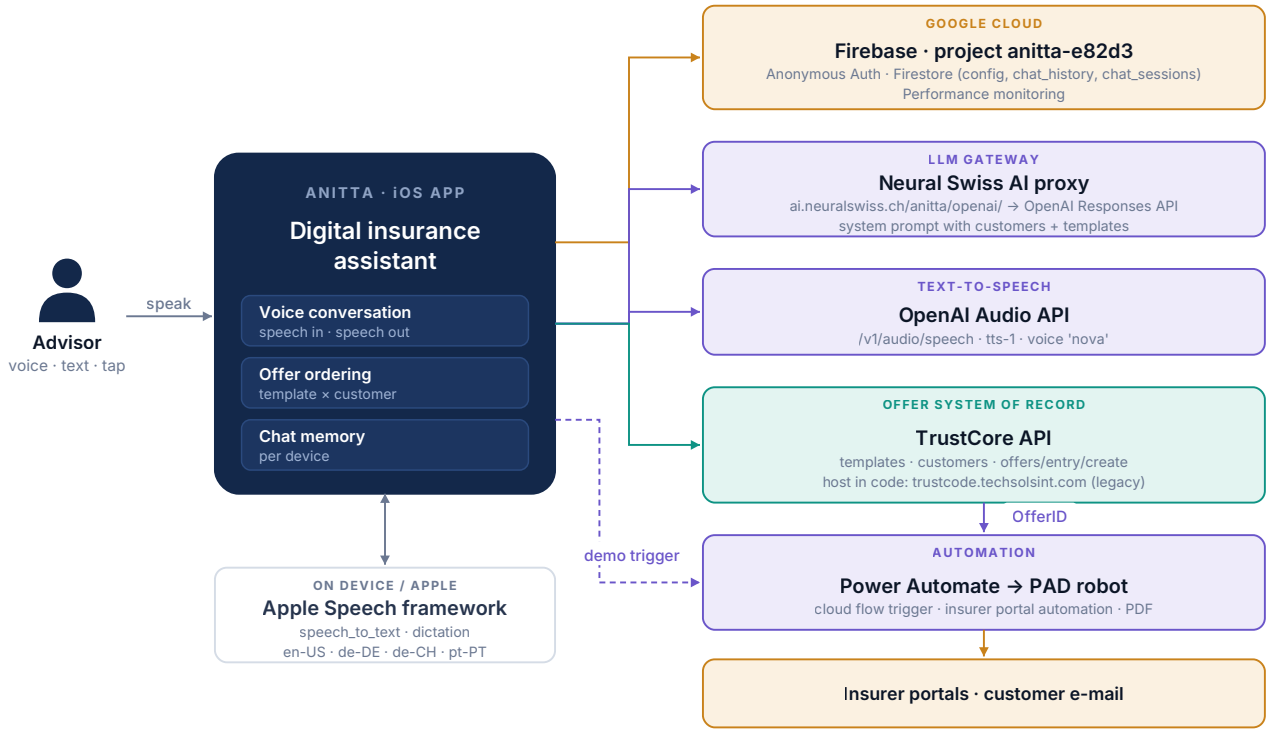


Figure 3 System context. The app on the left; cloud services on the right grouped by role. TrustCore and the automation chain below it are the same systems described in TC-TRM-001.

3.1 External services

SERVICE	INTERFACE	ROLE AND CONTRACT
Firebase Authentication	Anonymous sign-in at launch	Provides a per-installation uid used as the document id for conversations and as the `user` field sent to the LLM.
Cloud Firestore	Project `anitta-e82d3`	Runtime configuration (`config/global`) and conversation storage (`chat_history` , `chat_sessions`).
Firebase Performance	SDK	Automatic network and start-up traces.
Neural Swiss AI proxy	`POST ai.neuralswiss.ch/anitta/openai/`	Receives prompts and history, calls the OpenAI Responses API with the configured model, returns the Responses payload. Authenticated with a proxy key from configuration.
OpenAI Audio API	`POST api.openai.com/v1/audio/speech`	Text-to-speech with model `tts-1` , voice `nova` , speed 1; returns MP3. Called directly from the device.
Apple Speech	`speech_to_text` plugin 7.x	Dictation-mode recognition with partial results; server-assisted (on-device mode disabled).
TrustCore API	REST · Sanctum bearer token	Templates, customers and offer creation. The offer triggers TrustCore's Power Automate pipeline.

SERVICE	INTERFACE	ROLE AND CONTRACT
Power Automate	`workflow_url` from configuration	Demo trigger only; real offers reach the robot through TrustCore.

3.2 Relationship to TrustCore

TrustCore is Anitta's system of record. Anitta stores no offers, customers or templates of its own: it reads them from TrustCore on launch and writes offers back. Everything after offer creation — status changes, the robot run, the insurer portal, the one-time code relay, the PDF and the customer e-mail — is TrustCore's pipeline, unchanged. Consequently every TrustCore finding about offer automation (stuck offers, unauthenticated webhooks, the legacy callback host) applies equally to offers ordered through Anitta.

> **READING BOTH MANUALS**
> Chapter references of the form “TC 7.1” point to TC-TRM-001, the TrustCore Technical Reference Manual. Anitta-specific behaviour is documented here; shared behaviour is documented once, in TrustCore's manual.

4 Technology stack

A FlutterFlow-generated Flutter application with a hand-written layer for voice, LLM and TrustCore integration. Two state-management approaches coexist: FlutterFlow's Provider-based app state and a GetX controller.

LAYER	TECHNOLOGY	USE IN ANITTA
Language & framework	Dart ≥ 3.0, Flutter ≥ 3.24	Cross-platform UI; iOS is the shipping target (Android and web runners present)
App builder	FlutterFlow	Generated theme, widgets, navigation (GoRouter 12), Firestore schema helpers and API manager
State	Provider 6 (`FFAppState`) · GetX 4.7	App-wide settings and transcript · templates, customers and selection
Secure storage	`flutter_secure_storage` 9.2	Keychain storage for the speech key and the selected language
Firebase	firebase_core 3.8, auth 5.3, cloud_firestore 5.5, performance 0.10	Anonymous identity, configuration, conversation storage, telemetry
Voice in	`speech_to_text` 7 · `permission_handler` 11	Microphone and speech-recognition permissions and dictation
Voice out	`audioplayers` 6 · OpenAI `tts-1`	MP3 synthesis and playback
Networking	`http` 1.2.2	Direct calls to TrustCore, the proxy and OpenAI
UI	Lottie, flutter_animate, Google Fonts (Outfit, Figtree), fluttertoast	Waveform animation, page transitions, typography, toasts
Cloud Functions	Node 18 scaffold (FlutterFlow)	Private-API router with an empty call map; payment and push SDKs declared but unused

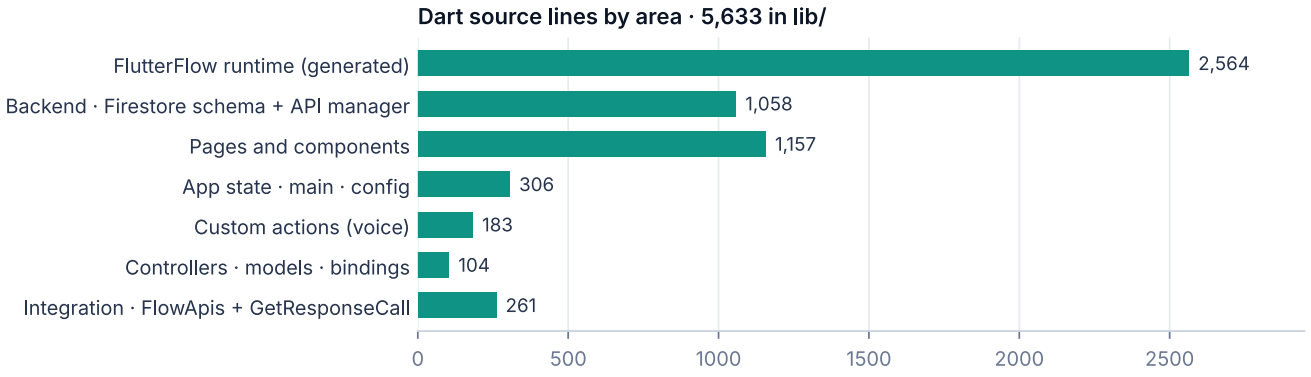


Figure 4 Dart source lines by area. Roughly half of the code is generated FlutterFlow runtime; the integration logic that defines Anitta's behaviour is compact.

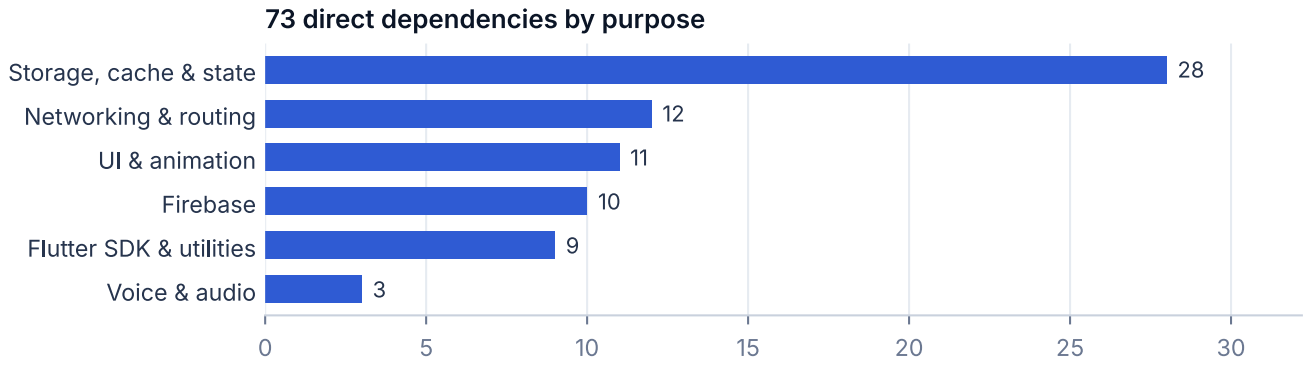


Figure 5 Direct dependencies by purpose. FlutterFlow pins each platform implementation explicitly, which inflates the count.

PART TWO

II

How Anitta Works

The four mechanisms behind the assistant: the conversational agent and its prompt, the voice loop, the hand-off of orders to TrustCore, and the memory that carries a conversation across sessions.

5 Conversational agent

6 Voice input and output

7 Ordering offers through TrustCore

8 Conversation memory and state

5 Conversational agent

Each turn is a single, stateless request to a language model that receives everything it needs in the prompt: instructions, language, the advisor's customers, the available templates, the full conversation so far and the new message.

5.1 Turn sequence

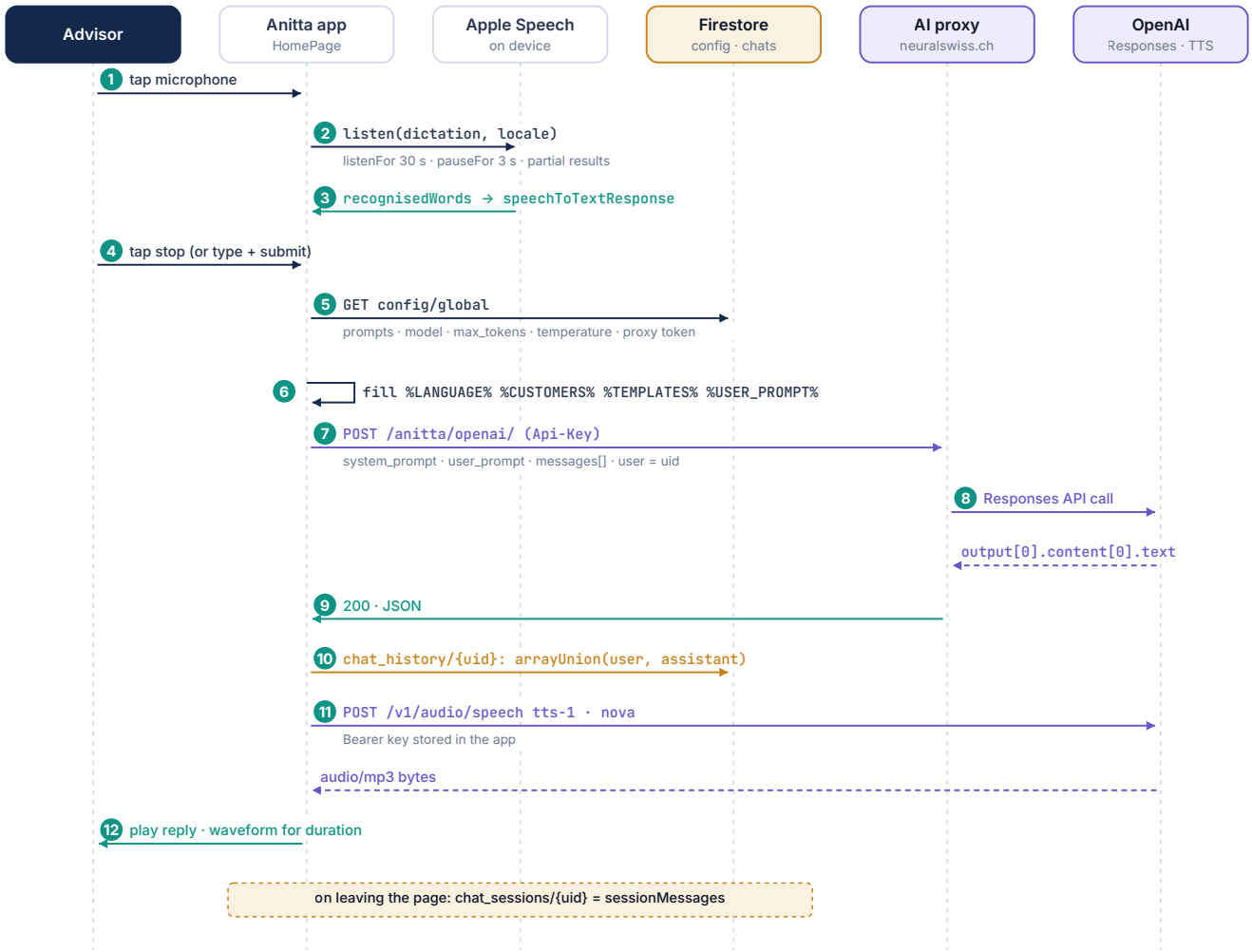


Figure 6 One conversational turn from microphone to spoken answer. Steps 5–10 are implemented in `HomePageWidget.getChatResponse()` and `GetResponseCall.call()`.

#	STEP	IMPLEMENTATION DETAIL
1–3	Capture	`startTextRecording()` initialises `SpeechToText`, maps the chosen language to a locale and streams recognised words into `FFAppState.speechToTextResponse`.
4	Submit	Stop button (or text field submission) calls `getChatResponse()`.
5	Configuration	Reads `config/global` on every turn: prompts, model, token limit, temperature and proxy key.
6	Prompt assembly	Placeholder substitution (section 5.2).

#	STEP	IMPLEMENTATION DETAIL
7–9	Inference	JSON POST to the AI proxy; the proxy calls the OpenAI Responses API; the answer is read from `$.output[0].content[0].text`.
10	Persist	User and assistant messages are appended to `chat_history/{uid}` with timestamps and to the in-memory session list.
11–12	Speak	`fetchSpeechAndPlay()` synthesises the answer, returns its duration and starts playback; a stopwatch drives the waveform for that duration.

5.2 Prompt assembly

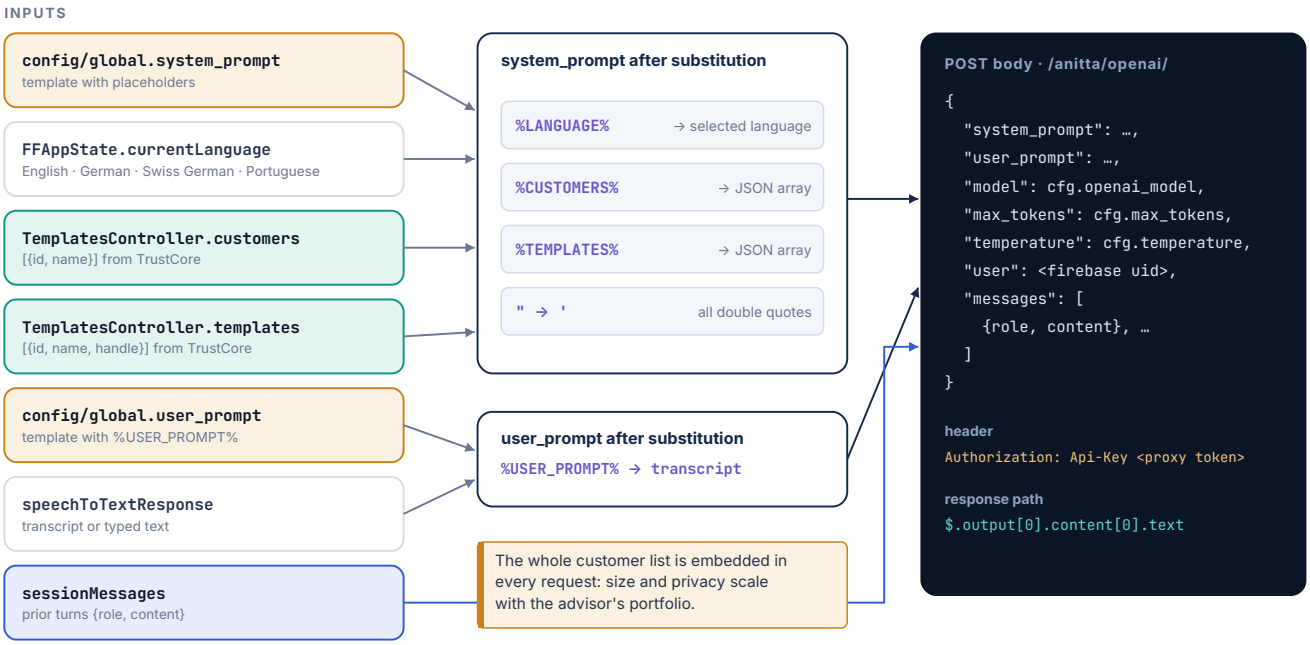


Figure 7 How the request to the model is assembled from configuration, app state, TrustCore data and conversation history.

The system prompt and user prompt are not in the code base: they are stored as templates in Firestore so they can be tuned without a release. The app substitutes four placeholders:

PLACEHOLDER	IN	REPLACED WITH
`%LANGUAGE%`	system_prompt	The selected voice language name, e.g. *Swiss German*
`%CUSTOMERS%`	system_prompt	JSON array `[{"id", "name"}]` of all customers loaded from TrustCore
`%TEMPLATES%`	system_prompt	JSON array `[{"id", "name", "handle"}]` of all templates loaded from TrustCore
`%USER_PROMPT%`	user_prompt	The transcript or typed text

After substitution every double quote in the system prompt is replaced by a single quote, so the embedded arrays are no longer valid JSON; models usually cope, but structured extraction from them is less reliable. The `language` argument passed to `GetResponseCall` is not used — the language comes from app state.

5.3 Model parameters

PARAMETER	SOURCE	NOTES
`model`	`config.openai_model`	Switchable without a release

PARAMETER	SOURCE	NOTES
`max_tokens`	`config.max_tokens`	Caps answer length and therefore speech duration
`temperature`	`config.temperature`	Lower values give more consistent answers about customers and templates
`user`	Firebase uid	Lets the proxy and provider attribute usage per installation
`messages`	session history	All previous turns of the current session, unbounded

5.4 What the agent can and cannot do today

CAPABILITY	STATUS	DETAIL
Answer questions about customers and templates	YES	Both lists are in the prompt
Converse in the advisor's language and remember context	YES	Language placeholder; session history
Speak its answer	YES	OpenAI TTS
Identify the template and customer the advisor means	MODEL	Possible in the answer text; not turned into an action by the app
Create the offer from the conversation	NOT IN APP	No tool or function call is parsed by the app. Unless the AI proxy performs actions server-side (not part of the delivered material), ordering is done in *My Templates*.
Report offer status	NO	No status call to TrustCore exists

Chapter 16 describes the tool-calling design that turns the conversation into a fully autonomous ordering agent.

6 Voice input and output

Speech recognition runs through Apple's framework; speech synthesis runs through OpenAI. The two are joined by app state and a stopwatch that keeps the on-screen waveform in step with the voice.

6.1 Speech recognition

SETTING	VALUE
Plugin	`speech_to_text` ^7.0.0 (Apple *SFSpeechRecognizer* on iOS)
Mode	`ListenMode.dictation`, partial results on, `cancelOnError: false`, `onDevice: false`
Timing	`listenFor` 30 s · `pauseFor` 3 s
Locale mapping	English → `en-US` · German → `de-DE` · Swiss German → `de-CH` · Portuguese → `pt-PT`; otherwise the device locale
Permissions	Microphone and speech recognition requested at app start; usage strings in `Info.plist`

IMPLEMENTATION NOTES

- `stopTextRecording()` creates a new `SpeechToText` instance and initialises it before stopping; it works because the plugin is a platform singleton, but the instance should be shared.
- The permission routine checks the microphone status twice instead of the speech-recognition status, so a denied speech permission is re-requested on every launch.
- Swiss German recognition (`de-CH`) produces Standard German text; dialect words may be normalised or missed.

6.2 Speech synthesis

SETTING	VALUE
Endpoint	`POST https://api.openai.com/v1/audio/speech`
Body	`{"model": "tts-1", "input": <answer>, "voice": "nova", "speed": "1"}`
Authentication	Bearer key from `FFAppState.apiKey`, whose default value is compiled into the app (risk A01)
Playback	`audioplayers` with MP3 bytes; the duration in milliseconds is returned and stored as `timerValue`
Failure	Non-200 throws; the turn is already saved, but no audio plays

Synthesis begins only after the full answer is received, so perceived latency is the sum of model latency and synthesis latency. Streaming the model output into a streaming TTS endpoint would let speech start after the first sentence.

7 Ordering offers through TrustCore

Ordering is the point where Anitta stops talking and starts working. A single POST creates the offer in TrustCore; TrustCore's automation then calculates and delivers it on the insurer's side.

7.1 End-to-end sequence

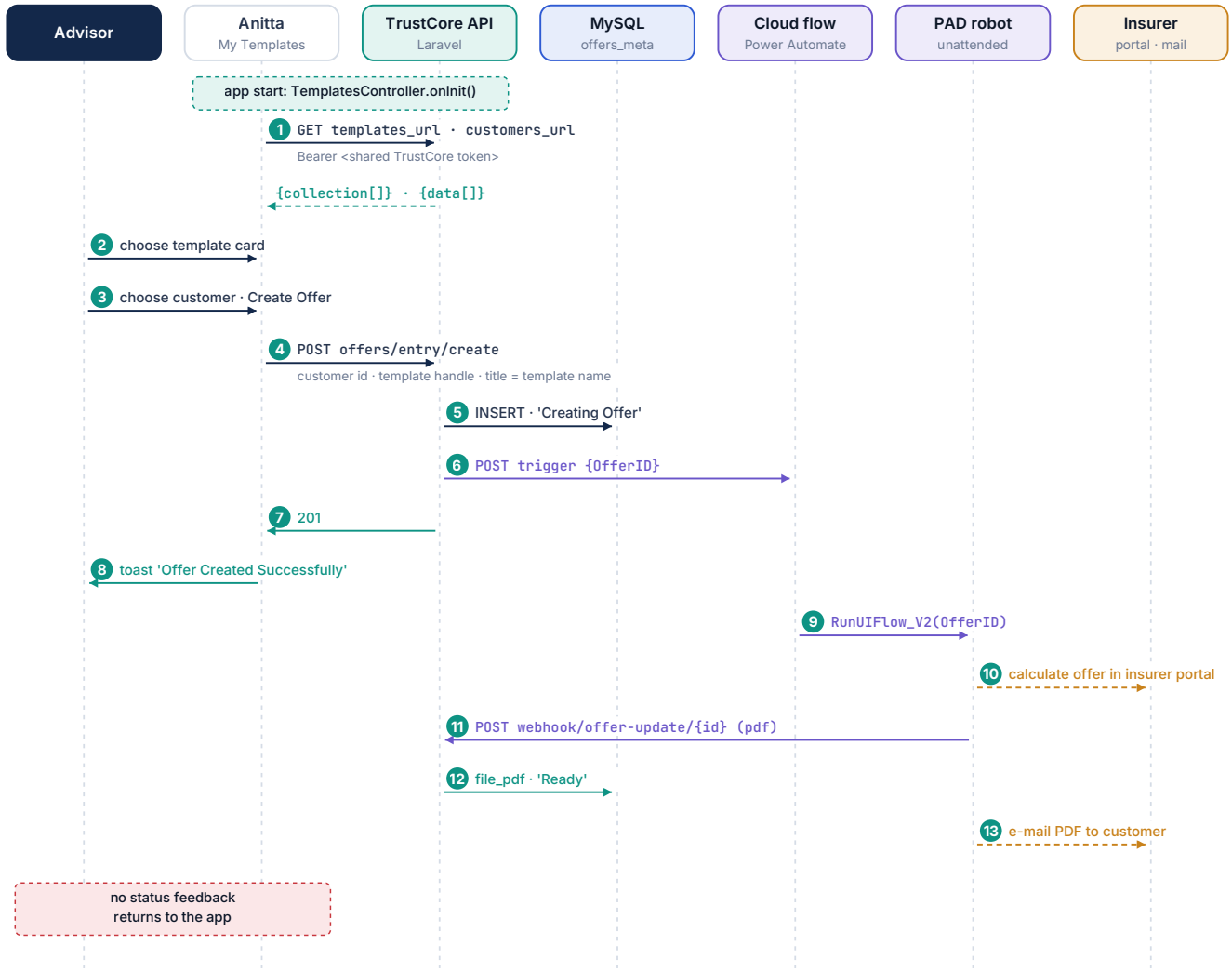


Figure 8 Ordering an offer from Anitta. Steps 1–8 are implemented in Anitta (`TemplatesController`, `FlowApis`); steps 5–13 are TrustCore's pipeline (TC 7.1). Dashed arrows run inside the automation.

7.2 Data loaded at launch

`CoreBindings` registers a permanent `TemplatesController`; its `onInit()` loads both lists in parallel without blocking the UI:

LIST	REQUEST	MAPPING
Templates	`GET {config.templates_url}`	`response.collection[]` → `Template{id, name, handle, status, description}`. A missing description is filled with the template description of TrustCore offer `20`, fetched from the legacy host.
Customers	`GET {config.customers_url}`	`response.data[]` → `Customer{id, name, email}`

The two URLs are configuration and are expected to point at TrustCore's `collections/all/templates` and `user/all` endpoints. Because Anitta sends neither `current_role` nor `current_user_id`, TrustCore applies its default scoping to these calls (TC 11.4): for `user/all` the default is the customer role filtered by an empty advisor id. The effective customer list therefore depends on the exact URL configured and should be verified.

7.3 Offer creation request

```
POST https://trustcode.techsolsint.com/trustcore-backend/public/api/collections/offers/entry/create
Authorization: Bearer <TrustCore token compiled into the app>
Content-Type: application/json

{ "rel_customer_req_col_users": <customer.id>,
  "rel_templates_req_col_collections": <template.handle>,
  "text_meta_title_req": <template.name> }

# 201 → toast "Offer Created Successfully" · anything else → "Failed to create offer"
```

Field mapping

TRUSTCORE COLUMN	ANITTA SOURCE	NOTES
`rel_customer_req_col_users`	`Customer.id`	TrustCore user id of role customer
`rel_templates_req_col_collections`	`Template.handle`	`template_*` handle; TrustCore reads the template's row 1 when the robot runs
`text_meta_title_req`	`Template.name`	Every offer for the same template gets the same title
`text_status`	— (set by TrustCore)	Forced to *Creating Offer*
`rel_created_by_col_users`	—	Not set; offers are not attributed to the advisor

7.4 What happens after the 201

TrustCore inserts the offer, triggers the Power Automate cloud flow with the new offer id and returns. The robot calculates the offer on the insurer portal (signing in with a one-time code relayed through TrustCore), uploads the PDF to TrustCore, which marks the offer *Ready*, and e-mails the PDF to the customer. On failure the flow posts *Failed* or *Timeout*. None of these events reach Anitta: the advisor sees only the initial toast.

> **WRONG TRUSTCORE HOST**
> The create and description calls are hard-coded to `trustcode.techsolsint.com`, the legacy TrustCore server. Unless that host proxies to production, offers ordered from Anitta are created on the legacy system, and — because TrustCore's flow trigger is hard-coded to the production flow — the production robot may then run against a production offer that shares the id (TC 3.3 and finding R10).

7.5 Demo automation

`FlowApis.demoAutomation()` posts an empty JSON request to `config.workflow_url` and treats 200, 201 and 202 as success. It is not connected to TrustCore and creates no offer. The result is not shown to the user.

8 Conversation memory and state

Anitta keeps two copies of each conversation in Firestore — a permanent history and a working session — plus a small amount of state on the device.

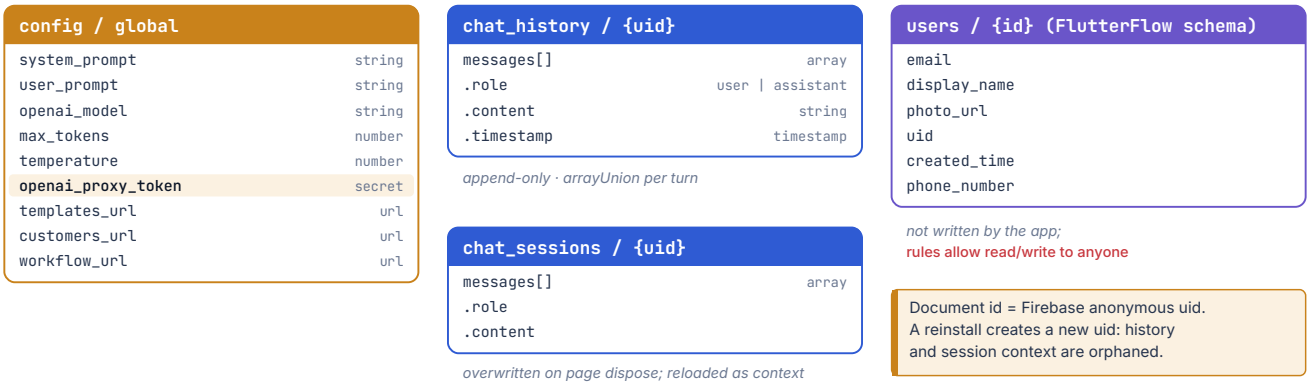


Figure 9 Firestore documents used by Anitta. All documents are keyed by the anonymous Firebase uid of the installation.

8.1 Two conversation stores

DOCUMENT	WRITTEN	READ	PURPOSE
`chat_history/{uid}`	after every successful turn, `arrayUnion` of two messages with timestamps	Chat History screen	Permanent transcript
`chat_sessions/{uid}`	when the home page is disposed, full overwrite	home page start	Context sent to the model on the next session

Because the session is written only on dispose, a turn made just before the app is killed is in the history but not in the next session's context. `arrayUnion` de-duplicates identical elements, but timestamps make every message unique, so the history grows without bound; Firestore's 1 MiB document limit will eventually reject writes for heavy users.

8.2 Device state

ITEM	WHERE	LIFETIME
`ff_currentLanguage`	Keychain	Persists across launches
`ff_apiKey`	Keychain, compiled default	Persists; never set by the UI
Transcript, timer value	`FFAppState` (memory)	Current turn
Templates, customers, selection	`TemplatesController` (memory)	Loaded once per launch; no refresh
Firebase uid	Firebase SDK	Until reinstall; a new uid orphans earlier conversations

PART THREE

III

Application and Interfaces

How the application is structured, every external call it makes, the configuration that steers it at runtime, and how identity flows from the advisor's device to TrustCore.

9	Application architecture
10	Interface reference
11	Runtime configuration
12	Identity and access

9 Application architecture

FlutterFlow generates the shell — theme, navigation, widgets, Firestore helpers and an API manager. Anitta's behaviour lives in a small hand-written layer: one GetX controller, two API classes and three custom actions.

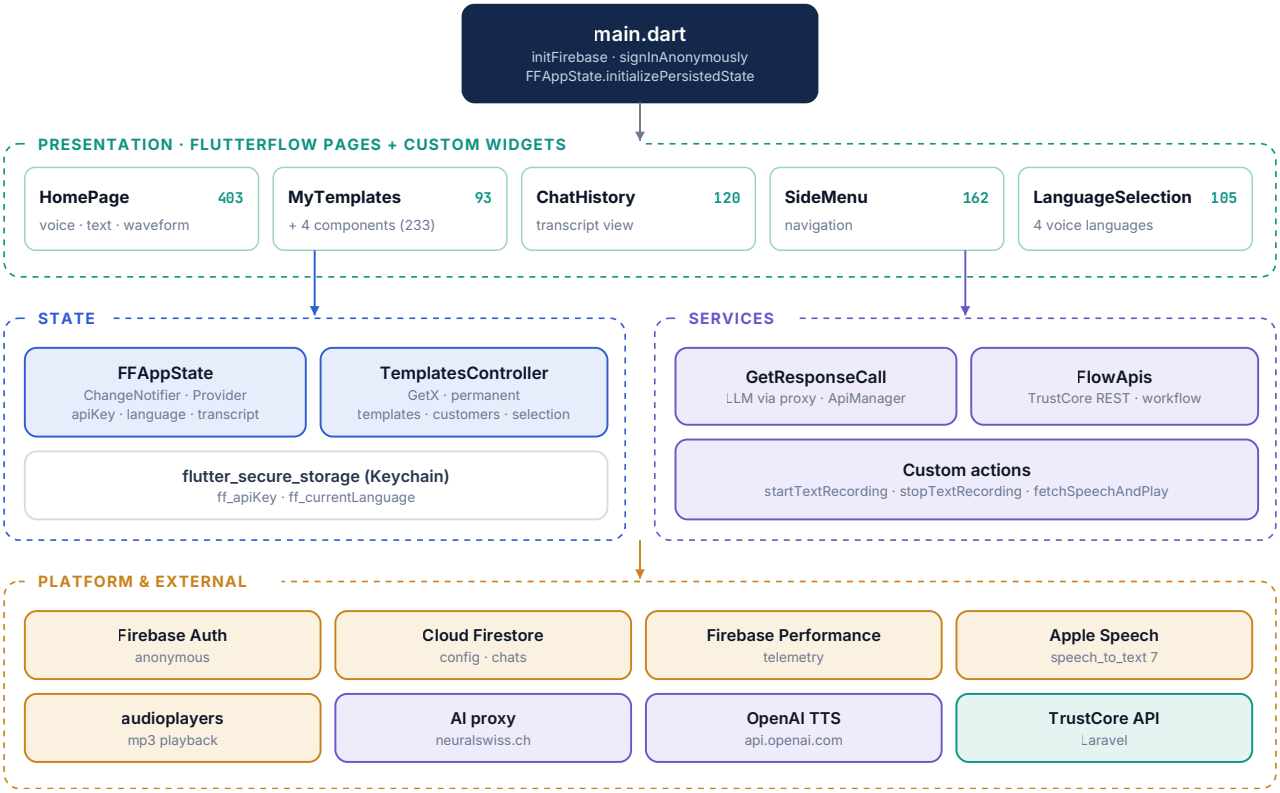


Figure 10 Layers of the application with the line counts of the hand-written pages.

9.1 Start-up

1. `main()` initialises Firebase, signs in anonymously and restores persisted app state from the Keychain.
2. `MyApp.initState()` requests microphone and speech permissions and builds the GoRouter.
3. `CoreBindings` registers `TemplatesController`, which starts loading templates and customers from TrustCore.
4. The router opens `HomePage`, which restores the previous session from `chat_sessions/{uid}`.

9.2 Navigation

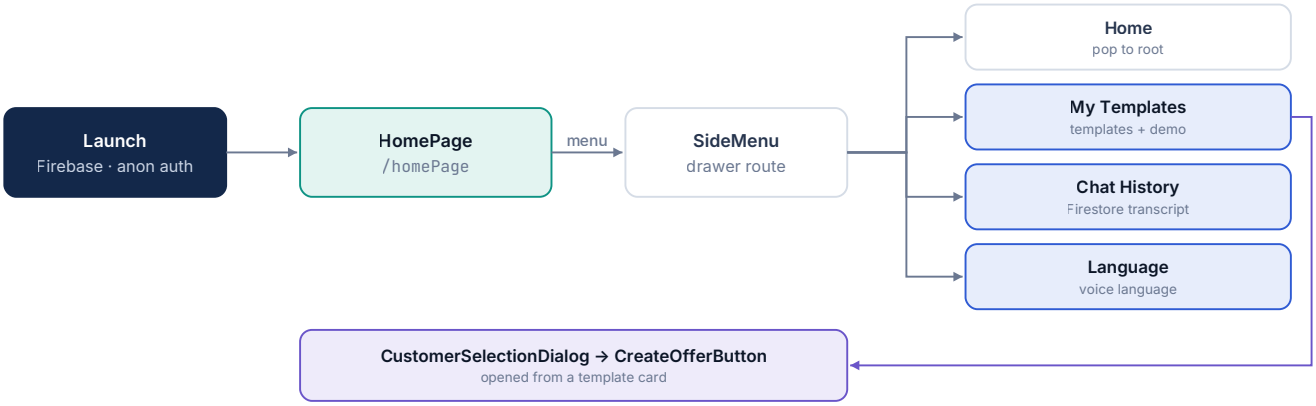


Figure 11 Screens and navigation. GoRouter defines the initial and home routes; the remaining screens are pushed imperatively from the side menu.

9.3 Component reference

COMPONENT	LINES	RESPONSIBILITY
`pages/home_page/home_page_widget.dart`	403	Voice and text input, LLM call, history persistence, TTS playback, waveform, reset
`pages/my_templates/` (+4 components)	326	Template cards, demo card, customer dialog, create-offer button
`pages/side_menu/side_menu.dart`	162	Drawer navigation
`pages/chat_history/chat_history.dart`	120	Transcript from `chat_history/{uid}`
`pages/language_selection/…`	105	Voice language picker
`backend/api_requests/flow_apis.dart`	144	TrustCore client and demo workflow trigger
`backend/api_requests/api_calls.dart`	117	`GetResponseCall` : prompt assembly and proxy call
`custom_code/actions/`	183	`startTextRecording` , `stopTextRecording` , `fetchSpeechAndPlay`
`controllers/templates_controller.dart`	48	Templates, customers, selection, `createOffer()`
`models/`	44	`Template` , `Customer` with `fromMap` and `toAPIJson`
`app_state.dart`	108	`FFAppState` : speech key, language, transcript, timer; Keychain persistence

9.4 Localisation

Two independent language settings exist. The **voice language** (English, German, Swiss German, Portuguese) drives speech recognition and the system prompt. The **interface locale** generated by FlutterFlow supports English, Spanish and French, and iOS declares `en` , `es` and `fr` localisations. Static screen texts added by hand (menu, titles, toasts) are English only. Aligning both sets — German and French interface for Swiss advisors — is recommended.

10 Interface reference

Every network call Anitta makes, with its authentication, payload and expected response.

VERB	ENDPOINT	AUTH	PAYLOAD → RESPONSE
POST	`ai.neuralswiss.ch/anitta/openai/`	`Api-Key` from config	`{system_prompt, user_prompt, model, max_tokens, temperature, user, messages[]}` → OpenAI Responses object
POST	`api.openai.com/v1/audio/speech`	Bearer (compiled)	`{model: tts-1, input, voice: nova, speed: 1}` → `audio/mpeg`
GET	`{config.templates_url}`	Bearer TrustCore (compiled)	→ `{collection: [{id, name, handle, status}]}`
GET	`{config.customers_url}`	Bearer TrustCore (compiled)	→ `{data: [{id, name, email, …}]}`
GET	`…/collections/offers/entry-detail/20?cleandata=true`	Bearer TrustCore (compiled)	→ `entry.rel_templates_col_collections.selected.textarea_template_description`
POST	`…/collections/offers/entry/create`	Bearer TrustCore (compiled)	`{rel_customer_req_col_users, rel_templates_req_col_collections, text_meta_title_req}` → 201
POST	`{config.workflow_url}`	none (signed URL)	empty → 200/201/202

TrustCore paths are prefixed with `https://trustcode.techsolsint.com/trustcore-backend/public/api/` where hard-coded. Configured URLs may point elsewhere.

10.1 Firestore operations

DOCUMENT	OPERATION	WHEN
`config/global`	get	Every conversational turn; launch (templates, customers); demo trigger
`chat_history/{uid}`	set (merge, arrayUnion)	After each successful turn
`chat_history/{uid}`	get · delete	Chat History screen · reset
`chat_sessions/{uid}`	get · set · delete	Home start · Home dispose · reset

10.2 Response parsing

The LLM answer is read with a fixed JSON path. If the proxy returns an error body, a refusal or a response whose first output item is not a message (for example a reasoning or tool item), the expression yields `null`, the stored assistant message is empty and speech synthesis receives the text “null”. Robust parsing should iterate over `output[]` for the first `message` item and handle non-200 responses explicitly.

11 Runtime configuration

One Firestore document, `config/global`, steers the assistant: what it says, which model it uses and which systems it talks to. Changes take effect on the next turn without an App Store release.

KEY	TYPE	USED BY AND EFFECT
`system_prompt`	string	Agent persona, rules and context; placeholders `%LANGUAGE%`, `%CUSTOMERS%`, `%TEMPLATES%`
`user_prompt`	string	Wrapper for the advisor's message; placeholder `%USER_PROMPT%`
`openai_model`	string	Model name forwarded by the proxy
`max_tokens`	number	Answer length limit
`temperature`	number	Sampling temperature
`openai_proxy_token`	secret	Credential for the AI proxy — readable by every client that can read the document
`templates_url`	url	TrustCore templates list
`customers_url`	url	TrustCore customers list
`workflow_url`	url	Demo automation trigger (likely a signed Power Automate URL)

RULES MISMATCH

The Firestore rules in the repository grant access only to `users/{document}` — to everyone, without authentication — and say nothing about `config`, `chat_history` or `chat_sessions`, which default rules would deny. Since the app depends on reading and writing those documents, the rules deployed in project `anitta-e82d3` must differ from the repository. They should be exported, reviewed and committed.

Recommended rules

```
match /config/{doc}        { allow read: if false; }                // move to the agent service
match /chat_history/{uid}  { allow read, write: if request.auth.uid == uid; }
match /chat_sessions/{uid} { allow read, write: if request.auth.uid == uid; }
match /users/{uid}         { allow read, write: if request.auth.uid == uid; }
```

12 Identity and access

Anitta has no notion of which advisor is using it. Every installation is an anonymous Firebase user, and every TrustCore call uses the same compiled token.

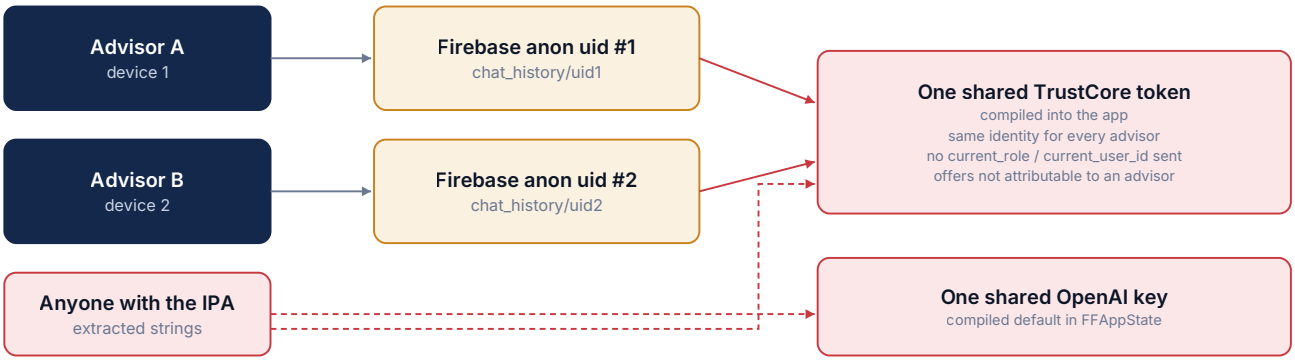


Figure 12 Identity today. Advisors are distinguished only by device; TrustCore and OpenAI see one shared identity, which is also available to anyone who extracts the app package.

12.1 Consequences

- **No attribution.** TrustCore cannot tell which advisor ordered an offer; its ownership and scoping rules (TC 11.4) do not apply.
- **No portfolio scoping.** All advisors see the same customer and template lists — whatever the shared token's account and the configured URLs return.
- **No revocation per person.** Removing one advisor's access means rotating the shared token and shipping a new build to everyone.
- **Device-bound memory.** Conversations follow the installation, not the person; a new phone starts from zero.

12.2 Target model

Advisors sign in with their TrustCore account (or an identity provider that TrustCore trusts). The agent service exchanges that identity for a short-lived, advisor-scoped TrustCore token and holds all third-party keys. The app holds only the advisor's session. Chapter 16 places this in the roadmap.

PART FOUR

IV

Security, Operations and Roadmap

A risk-ranked review of the application, the state of the code, how builds are produced and released, and the design and plan that turn Anitta into a fully autonomous ordering agent.

13 Security assessment

14 Code quality and technical debt

15 Build and release

16 Towards a fully autonomous agent

13 Security assessment

Sixteen risks were identified by reading every Dart file, the iOS project, the Firebase rules and functions, and by tracing each call into TrustCore. The two critical findings are secrets compiled into the app.

13.1 Method

White-box review of repository `anitta-main` at version 0.7.2+15. Mobile binaries can be unpacked with freely available tools, so any string compiled into the app is treated as public. Ratings follow the scheme of TC-TRM-001: impact on confidentiality, integrity and availability; likelihood based on exposure and required knowledge. Firestore findings assume the rules in the repository unless the deployed rules are shown to differ.

13.2 Risk heat map

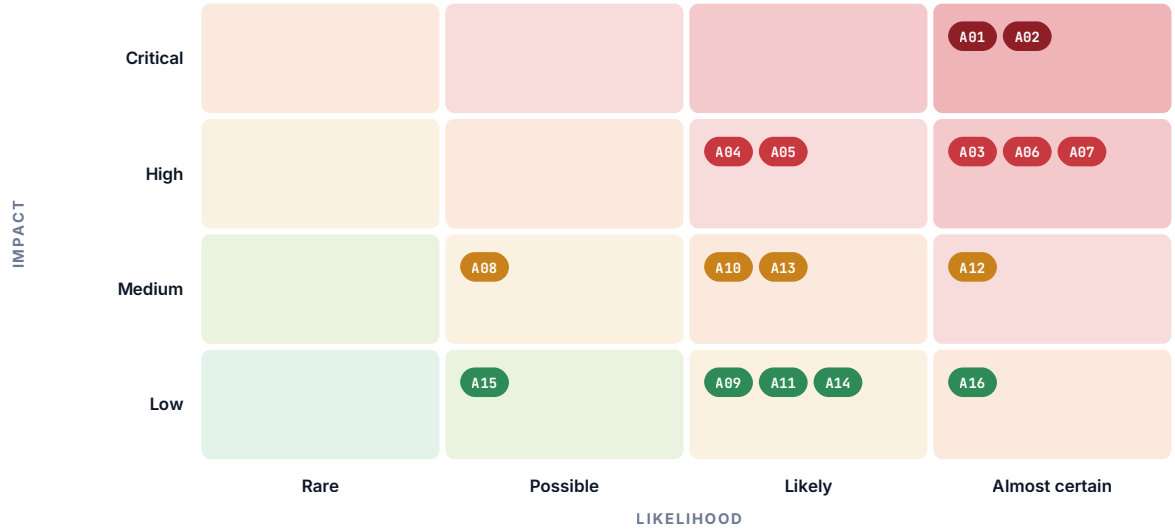


Figure 16 Distribution of the sixteen findings by impact and likelihood.

13.3 Summary

ID	FINDING	AREA	IMPACT	LIKELIHOOD
A01	OpenAI secret key compiled into the app	Secrets	CRITICAL	Almost certain
A02	TrustCore API token compiled into the app	Secrets	CRITICAL	Almost certain
A03	App targets the legacy TrustCore host	Integration	HIGH	Almost certain
A04	Firestore rules open or undocumented	Data	HIGH	Likely
A05	Proxy credential and workflow URL readable by clients	Secrets	HIGH	Likely
A06	No advisor identity or attribution	Identity	HIGH	Almost certain
A07	Customer data sent to a third-party model	Privacy	HIGH	Almost certain
A08	No guardrails for future autonomous actions	Agent safety	MEDIUM	Possible
A09	Demo automation visible to every advisor	Operations	LOW	Likely

ID	FINDING	AREA	IMPACT	LIKELIHOOD
A10	Unbounded conversation retention	Privacy	MEDIUM	Likely
A11	Fragile response parsing	Reliability	LOW	Likely
A12	No offer status feedback	Product	MEDIUM	Almost certain
A13	Customer list depends on TrustCore default scoping	Integration	MEDIUM	Likely
A14	Speech permission check defect	Reliability	LOW	Likely
A15	Unused SDKs in Cloud Functions scaffold	Supply chain	LOW	Possible
A16	Template description taken from a fixed legacy offer	Integration	LOW	Almost certain

13.4 Findings

A01 OpenAI secret key compiled into the app — CRITICAL

CONTEXT Secrets · Almost certain

EVIDENCE `lib/app_state.dart`: default value of `FFAppState._apiKey` is a full `sk-proj-…` key; used as Bearer token by `fetchSpeechAndPlay`.

IMPACT Anyone who unpacks the IPA can use the organisation's OpenAI account without limit (cost, quota exhaustion, abuse attributed to the organisation). The key is also in source control history.

REMEDIATION Revoke the key immediately in the OpenAI dashboard; move speech synthesis behind the agent service; scan the repository history and purge.

A02 TrustCore API token compiled into the app — CRITICAL

CONTEXT Secrets · Almost certain

EVIDENCE `lib/backend/api_requests/flow_apis.dart`: Sanctum token `85|…` in four request headers.

IMPACT Combined with TrustCore findings R02/R03, the token allows reading customer data, creating offers that start the robot, and modifying user accounts — from any machine, for as long as the token lives (TrustCore tokens never expire).

REMEDIATION Delete token 85 in `personal_access_tokens`; issue advisor-scoped, expiring tokens from the agent service; never ship TrustCore credentials in a client.

A03 App targets the legacy TrustCore host — HIGH

CONTEXT Integration · Almost certain

EVIDENCE `flow_apis.dart`: offer creation and template description use `https://trustcode.techsolsint.com/…`.

IMPACT Offers ordered in Anitta may be created on the legacy server, invisible in production; combined with TrustCore's hard-coded production flow trigger, the robot may act on an unrelated production offer with the same id.

REMEDIATION Move all TrustCore base URLs to configuration per environment; point production builds at `trustcore.ch`; verify existing offers.

A04 Firestore rules open or undocumented — HIGH

CONTEXT	Data · Likely
EVIDENCE	`firebase/firestore.rules`: `users/{document}` allows create, read, write and delete to everyone; no rules for `config`, `chat_history`, `chat_sessions`.
IMPACT	If the deployed rules match the repository, the app cannot work; if they were opened to make it work, every conversation (with customer details) may be readable by any client of the project.
REMEDIATION	Export and commit the deployed rules; restrict chats to `request.auth.uid == uid`; deny client access to `config`.

A05 Proxy credential and workflow URL readable by clients — HIGH

CONTEXT	Secrets · Likely
EVIDENCE	`config/global.openai_proxy_token` and `workflow_url` are fetched by the app on every turn.
IMPACT	Any anonymous Firebase client of the project can read the proxy key (LLM usage on the organisation's account) and the automation trigger URL.
REMEDIATION	Move configuration to the agent service; rotate the proxy key and the workflow URL signature.

A06 No advisor identity or attribution — HIGH

CONTEXT	Identity · Almost certain
EVIDENCE	`main.dart`: `signInAnonymously()`; all TrustCore calls use one token; no `current_user_id`.
IMPACT	Offers cannot be traced to the advisor who ordered them; advisor-level scoping in TrustCore is bypassed; access cannot be revoked per person.
REMEDIATION	Advisor sign-in; per-advisor TrustCore tokens via the agent service; audit log of orders.

A07 Customer data sent to a third-party model — HIGH

CONTEXT	Privacy · Almost certain
EVIDENCE	`api_calls.dart`: `%CUSTOMERS%` replaced with all customer ids and names in every request; conversation history included.
IMPACT	Personal data of every customer in the portfolio is disclosed to the proxy and the model provider on every turn, regardless of relevance; nFADP obligations apply.
REMEDIATION	Search customers on demand via tools; pseudonymise; contract and configure zero-retention or Swiss/EU hosting; inform customers.

A08 No guardrails for future autonomous actions — MEDIUM

CONTEXT	Agent safety · Possible
EVIDENCE	Customer and template names are injected verbatim into the system prompt; no confirmation layer exists.
IMPACT	Once the model can act, a crafted customer or template name could steer it (prompt injection), and misrecognised speech could order the wrong offer.
REMEDIATION	Typed tools with server-side validation; explicit spoken confirmation before `create_offer`; treat data fields as untrusted.

A09 Demo automation visible to every advisor — LOW

CONTEXT	Operations · Likely
EVIDENCE	`my_templates.dart`: *Group Mutuel Demo Offer* card calls `FlowApis.demoAutomation()`.
IMPACT	Advisors can start demo runs on the shared robot, consuming capacity and confusing statistics.
REMEDIATION	Hide behind a feature flag or a demo build.

A10 Unbounded conversation retention — MEDIUM

CONTEXT	Privacy · Likely
EVIDENCE	`chat_history/{uid}` grows by two messages per turn; no deletion except manual reset.
IMPACT	Personal data kept indefinitely; documents approach the 1 MiB Firestore limit; history orphaned on reinstall.
REMEDIATION	Retention period (e.g. 90 days), per-advisor storage, sub-collection per message.

A11 Fragile response parsing — LOW

CONTEXT	Reliability · Likely
EVIDENCE	`$.output[0].content[0].text` assumed; error bodies and non-message items not handled.
IMPACT	Empty or “null” answers are stored and spoken.
REMEDIATION	Parse the first message item; handle non-200 and refusals with a spoken fallback.

A12 No offer status feedback — MEDIUM

CONTEXT	Product · Almost certain
EVIDENCE	No call to TrustCore after creation; toast only.
IMPACT	Failed or stuck offers (21 % failure rate in TrustCore at export) go unnoticed by the advisor until the customer complains.
REMEDIATION	Status polling or webhooks into the agent service; push notifications using the existing entitlement.

A13 Customer list depends on TrustCore default scoping — MEDIUM

CONTEXT	Integration · Likely
EVIDENCE	No role parameters sent; TrustCore `user/all` falls back to customers filtered by an empty advisor id.
IMPACT	Advisors may see too few or too many customers depending on the configured URL.
REMEDIATION	Explicit advisor-scoped endpoint in TrustCore; verify the configured URL.

A14 Speech permission check defect — LOW

CONTEXT	Reliability · Likely
EVIDENCE	`main.dart`: `speechRecognitionStatus` reads `Permission.microphone`.
IMPACT	Speech permission state is misreported; prompts may repeat.
REMEDIATION	Check `Permission.speech`.

A15 Unused SDKs in Cloud Functions scaffold — LOW

CONTEXT	Supply chain · Possible
EVIDENCE	`firebase/functions/package.json`: Stripe, Braintree, Razorpay, Mux, OneSignal; empty call map.
IMPACT	Larger attack and maintenance surface if functions are deployed.
REMEDIATION	Remove unused dependencies or the functions folder.

A16 Template description taken from a fixed legacy offer — LOW

CONTEXT	Integration · Almost certain
EVIDENCE	`FlowApis.getDescription()` reads offer 20 on the legacy host for every template without description.
IMPACT	Misleading descriptions; extra dependency on the legacy server at every launch.
REMEDIATION	Read descriptions from the template itself in production TrustCore.

13.5 Data protection

Each request to the language model contains the names and TrustCore ids of all customers available to the app, plus the conversation, which may include health-insurance details spoken by the advisor. The request passes through the Neural Swiss proxy to OpenAI. Under the Swiss nFADP this is a disclosure of personal data to a processor, possibly abroad, and requires a processing agreement, information to customers and an assessment of the provider's location and retention. Conversation documents in Firestore are personal data with no retention limit. Recommended controls: pseudonymous customer references in prompts, zero-retention or Swiss-hosted model endpoints, a retention period for `chat_history`, and deletion on request.

14 Code quality and technical debt

The app achieves a convincing voice experience with little code. The trade-off is a prototype structure: secrets in the client, two state systems, hard-coded endpoints and no tests.

14.1 Strengths

- **Remote-tunable behaviour.** Prompts, model and parameters live in configuration and can be changed without a release.
- **Small integration surface.** All TrustCore and LLM traffic passes through two classes, which makes the planned move to a backend straightforward.
- **Natural interaction.** Dictation with partial results, spoken answers and a duration-synchronised waveform.
- **Clean separation of ordering.** Offer creation is one method with one request, mirroring TrustCore's own form.

14.2 Debt register

#	ITEM	DETAIL	EFFORT
D1	Secrets and hosts in code	OpenAI key, TrustCore token and legacy host compiled into Dart sources.	Medium
D2	Two state frameworks	FlutterFlow Provider state and GetX controller side by side; GetX is outside FlutterFlow's editor, so regenerating the project can overwrite integration code.	Medium
D3	Configuration read per turn	`config/global` fetched on every request and three times at launch; cache with a listener instead.	Small
D4	Fragile response parsing	Fixed JSON path into the Responses payload; no handling of tool or reasoning items.	Small
D5	No refresh of TrustCore data	Templates and customers are loaded once per app launch; new customers need an app restart.	Small
D6	Unbounded context	Full session history and full customer list in every request; cost and latency grow with use.	Medium
D7	Generated scaffolding	Cloud Functions package declares Stripe, Braintree, Razorpay, Mux and OneSignal; none is used.	Small
D8	Speech instance handling	New `SpeechToText` per start and stop; permission check bug.	Small
D9	No tests or CI	No widget, integration or golden tests; no pipeline.	Large
D10	Static demo in production UI	Demo card visible to every advisor.	Small
D11	Hard-coded description source	Default template description taken from TrustCore offer 20.	Small
D12	Push capability unused	`aps-environment` entitlement and an `ImageNotification` service extension exist but no notifications are sent.	Small

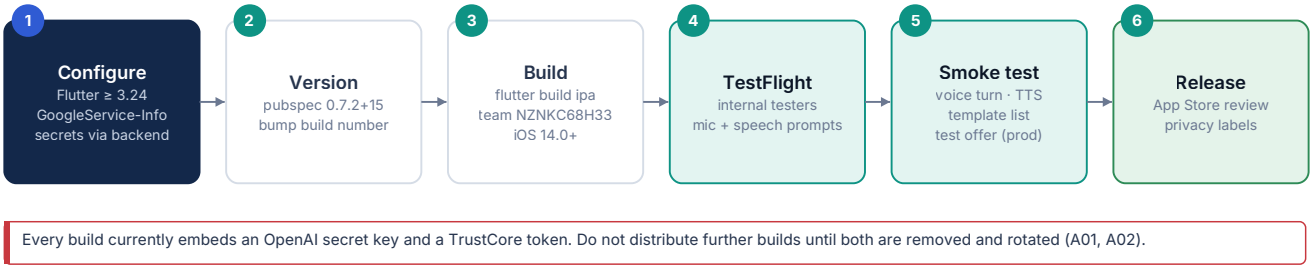
15 Build and release

Build facts, the release procedure, and the checks that confirm a build is healthy.

15.1 Build facts

ITEM	VALUE
Bundle identifier	`ch.neotralo.anitta` · display name *Anitta*
Version	`0.7.2+15` (pubspec)
Minimum iOS	14.0
Apple team	`NZNKC68H33`
Toolchain	Flutter ≥ 3.24 (stable), Dart ≥ 3.0, Xcode with CocoaPods
Firebase project	`anitta-e82d3` (iOS, Android and web apps registered)
Entitlements	Push notifications (`aps-environment`); notification service extension `ImageNotification`
Privacy strings	Microphone and speech-recognition usage descriptions

15.2 Release procedure



Every build currently embeds an OpenAI secret key and a TrustCore token. Do not distribute further builds until both are removed and rotated (A01, A02).

Figure 13 Release steps from configuration to App Store.

```
flutter pub get
cd ios && pod install && cd ..
flutter build ipa --release                  # signs with team NZNKC68H33
# upload build/ios/ipa/*.ipa with Transporter or Xcode Organizer → TestFlight
```

15.3 Smoke test

1. First launch asks for microphone and speech permissions.
2. A spoken question in each voice language is transcribed correctly and answered aloud.
3. *My Templates* lists the expected templates; the customer dialog lists the advisor's customers.
4. In production only: a test offer is created, appears in TrustCore as *Creating Offer* and reaches *Ready*.
5. *Chat History* shows the turns; reset clears them.

15.4 Operational checks

CHECK	FREQUENCY	HOW
LLM and TTS cost	Weekly	Provider usage dashboards per key; alert on spikes (a leaked key shows here first)
Firestore document sizes	Monthly	Largest `chat_history` documents against the 1 MiB limit
Offer success rate	Weekly	TrustCore offers created by the shared account: Ready vs Failed vs stuck
Crash and performance	Weekly	Firebase Performance traces; add Crashlytics

16 Towards a fully autonomous agent

The product promise is that an advisor says “prepare a Groupe Mutuel offer for Anna Müller with hospital and dental cover” and Anitta does the rest. The building blocks exist; this chapter connects them safely.

16.1 Target architecture

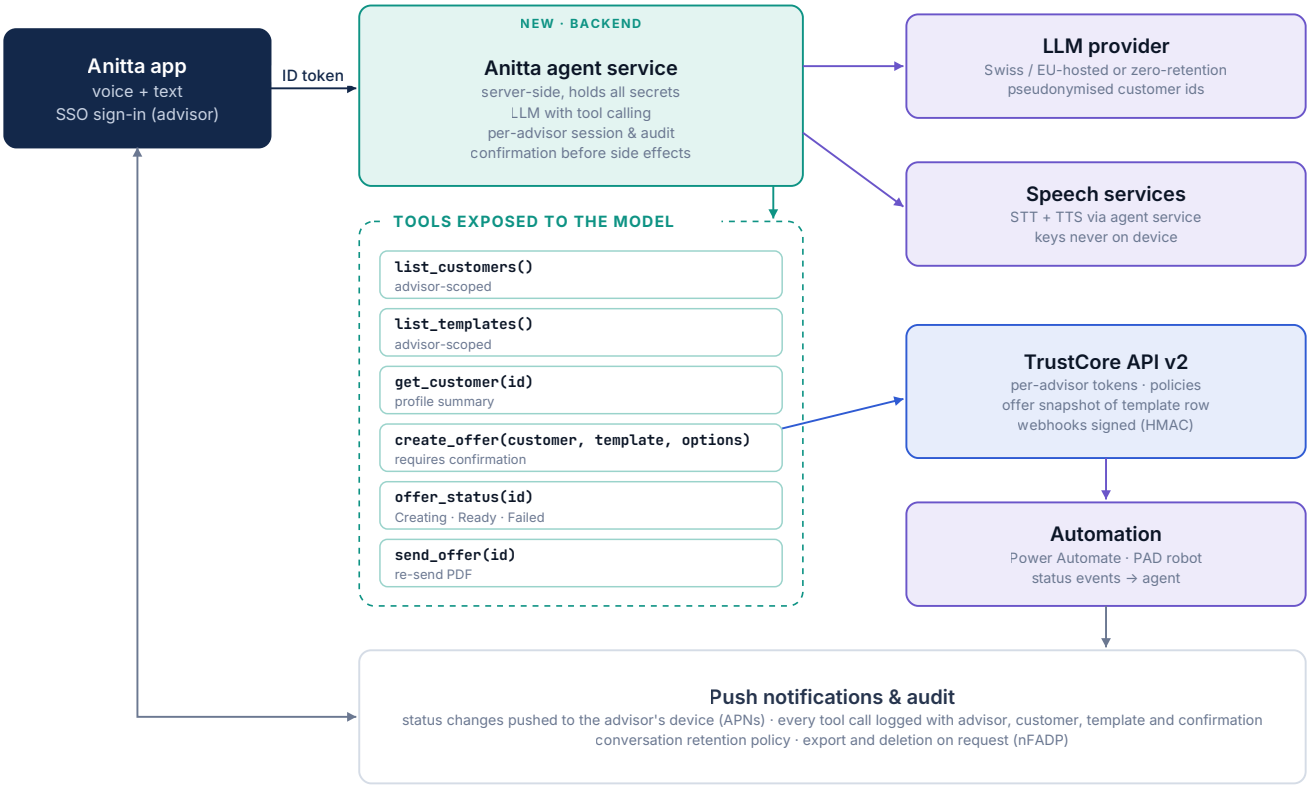


Figure 14 Target architecture. A server-side agent service holds all secrets, authenticates advisors, exposes TrustCore operations as tools to the model and reports status back to the device.

16.2 Design principles

PRINCIPLE	APPLICATION
No secrets on the device	The app talks only to the agent service with the advisor's session; LLM, TTS and TrustCore credentials stay on the server.
Act through tools, not text	The model calls typed tools (`list_customers` , `create_offer` …); the service validates arguments against TrustCore before acting.
Confirm before side effects	`create_offer` returns a summary that Anitta reads back; only an explicit “yes” executes it. This guards against misrecognised names and prompt injection.
Least data to the model	Customers are retrieved on demand by search, not dumped into every prompt; ids are pseudonymised.
Closed loop	TrustCore status changes (Ready, Failed, Timeout) reach the agent and the advisor by push notification, using the entitlement already in the app.

PRINCIPLE	APPLICATION
Auditability	Every tool call is logged with advisor, customer, template, confirmation and result.

16.3 Conversation flow with tools

```
advisor  "Prepare a Nur KVG offer for Anna Müller."
model  → list_customers(query="Anna Müller")      ⇒ 1 match, id 18
model  → list_templates(query="Nur KVG")          ⇒ template_nur_kvg
model    "I'll create a Nur KVG offer for Anna Müller. Shall I go ahead?"
advisor  "Yes."
model  → create_offer(customer=18, template="template_nur_kvg", confirmed=true)
service  POST TrustCore offers/entry/create (advisor token) ⇒ 201, offer 53
model    "Done. Offer 53 is being calculated; I'll let you know when the PDF is sent."
push     "Offer 53 for Anna Müller is ready and was e-mailed."   ← TrustCore status webhook
```

16.4 Prerequisites in TrustCore

The autonomous agent depends on TrustCore fixes from TC-TRM-001: server-side roles and policies (R03) so advisor tokens are scoped; authenticated webhooks (R01, R09) so status events can be trusted; a per-environment flow trigger (R10) so the agent can be tested on staging; and template snapshots in offers so what the advisor confirmed is what the robot builds.

16.5 Roadmap

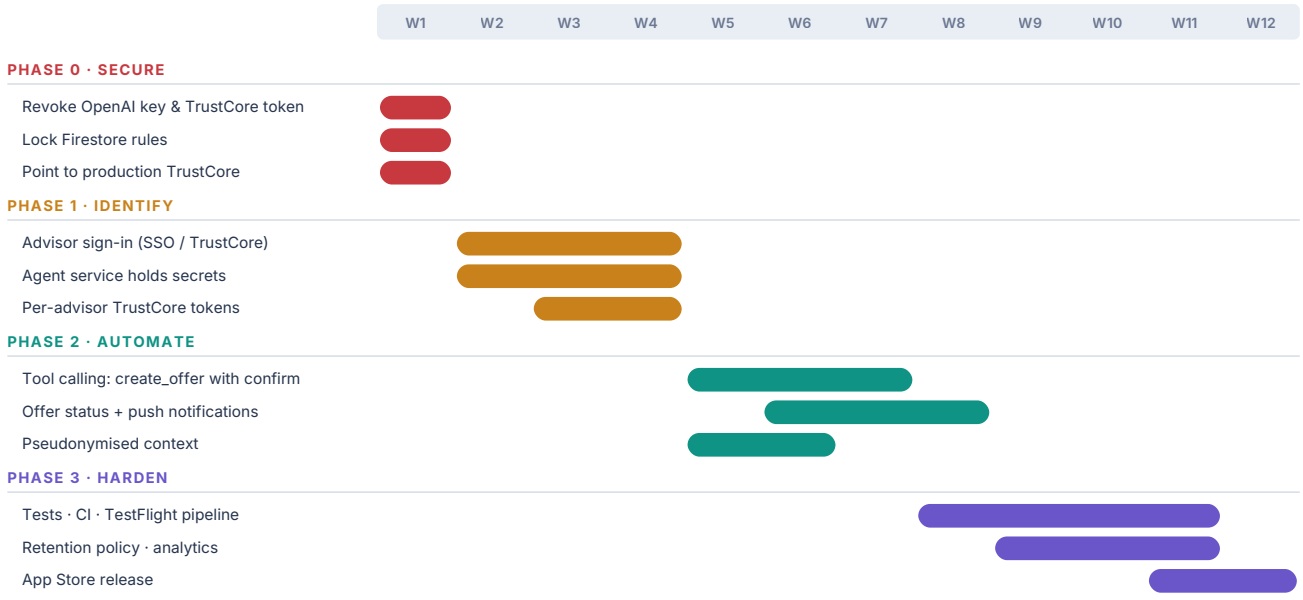


Figure 15 Twelve-week plan in four phases, aligned with the TrustCore remediation plan.

PHASE	DELIVERABLES	CLOSES
0 · Secure (W1)	Revoke and rotate the OpenAI key and the TrustCore token; remove both from the code; lock Firestore rules; point the app at production TrustCore; hide the demo card.	A01–A05, A09
1 · Identify (W2–W4)	Advisor sign-in; agent service holding secrets and calling LLM/TTS; per-advisor TrustCore tokens; conversations keyed by advisor.	A06, A07, A10

PHASE	DELIVERABLES	CLOSES
2 · Automate (W5–W8)	Tool calling with confirmation; on-demand customer search; offer status and push notifications; streaming speech.	A08, A12, A13
3 · Harden (W8–W12)	Tests and CI with TestFlight distribution; retention policy; Crashlytics; App Store release with privacy labels.	A11, A14–A16

APPENDICES

A

Reference Data

Source inventory, data schemas and payloads, the cross-reference to TrustCore findings, and a glossary.

A	Source inventory
B	Payload and data schemas
C	TrustCore cross-reference
D	Glossary

A Source inventory

Every Dart file in `lib/` with its origin and size. “Generated” files come from FlutterFlow; hand-written files carry Anitta's behaviour.

FILE	ORIGIN	LINES
`lib/backend/api_requests/api_manager.dart`	generated	520
`lib/flutter_flow/flutter_flow_util.dart`	generated	451
`lib/pages/home_page/home_page_widget.dart`	hand-written	403
`lib/flutter_flow/flutter_flow_theme.dart`	generated	305
`lib/flutter_flow/flutter_flow_widgets.dart`	generated	291
`lib/flutter_flow/internationalization.dart`	generated	289
`lib/flutter_flow/nav/serialization_util.dart`	generated	272
`lib/flutter_flow/nav/nav.dart`	generated	242
`lib/backend/backend.dart`	generated	186
`lib/flutter_flow/flutter_flow_model.dart`	generated	172
`lib/flutter_flow/flutter_flow_icon_button.dart`	generated	170
`lib/pages/side_menu/side_menu.dart`	hand-written	162
`lib/backend/schema/util/firestore_util.dart`	generated	150
`lib/backend/api_requests/flow_apis.dart`	hand-written	144
`lib/backend/schema/users_record.dart`	generated	137
`lib/flutter_flow/flutter_flow_timer.dart`	generated	128
`lib/main.dart`	hand-written	121
`lib/pages/chat_history/chat_history.dart`	hand-written	120
`lib/backend/api_requests/api_calls.dart`	hand-written	117
`lib/flutter_flow/flutter_flow_animations.dart`	generated	111
`lib/app_state.dart`	hand-written	108
`lib/pages/language_selection/language_selection.dart`	hand-written	105
`lib/pages/my_templates/my_templates.dart`	hand-written	93
`lib/pages/my_templates/components/offer_card.dart`	hand-written	85
`lib/custom_code/actions/fetch_speech_and_play.dart`	hand-written	79
`lib/firebase_options.dart`	generated	75
`lib/custom_code/actions/start_text_recording.dart`	hand-written	73
`lib/flutter_flow/uploaded_file.dart`	generated	68
`lib/pages/my_templates/components/customer_card.dart`	hand-written	67
`lib/pages/my_templates/components/customer_selection_dialog.dart`	hand-written	55
`lib/controllers/templates_controller.dart`	hand-written	48
`lib/flutter_flow/place.dart`	generated	46
`lib/pages/home_page/home_page_model.dart`	hand-written	41
`lib/backend/schema/util/schema_util.dart`	generated	40
`lib/custom_code/actions/stop_text_recording.dart`	hand-written	28

FILE	ORIGIN	LINES
`lib/pages/my_templates/components/create_offer_button.dart`	hand-written	26
`lib/models/template.dart`	hand-written	26
`lib/flutter_flow/lat_lng.dart`	generated	19
`lib/models/customer.dart`	hand-written	18
`lib/backend/firebase/firebase_config.dart`	hand-written	18
`lib/bindings/core_bindings.dart`	hand-written	12
`lib/backend/api_requests/get_streamed_response.dart`	hand-written	4
`lib/custom_code/actions/index.dart`	hand-written	3
`lib/backend/schema/index.dart`	generated	3
`lib/index.dart`	hand-written	2

B Payload and data schemas

Exact shapes of every request Anitta sends and every document it stores.

LLM request (app → AI proxy)

```
{ "system_prompt": string,      // placeholders substituted, " replaced by '
  "user_prompt":   string,
  "model":         string,      // config.openai_model
  "max_tokens":    number,
  "temperature":   number,
  "user":          string,      // Firebase anonymous uid
  "messages":      [ { "role": "user" | "assistant", "content": string } ] }
```

LLM response (consumed fields)

```
{ "output": [ { "content": [ { "text": string } ] } ] }    // $.output[0].content[0].text
```

Speech synthesis request (app → OpenAI)

```
{ "model": "tts-1", "input": string, "voice": "nova", "speed": "1" }   → audio/mpeg
```

Offer creation (app → TrustCore)

```
{ "rel_customer_req_col_users": int,
  "rel_templates_req_col_collections": string,   // template handle
  "text_meta_title_req": string }              → 201 { message, id }
```

Conversation message (Firestore)

```
{ "role": "user" | "assistant", "content": string, "timestamp": Timestamp }   // timestamp only in
chat_history
```

App models

```
Template { id: int, name: String, description: String?, handle: String, status: String? }   toAPIJson →
{id, name, handle}
Customer { id: int, name: String, email: String }                                          toAPIJson → {id,
name}
```

C TrustCore cross-reference

Anitta findings whose impact depends on TrustCore findings documented in TC-TRM-001. Fixing the TrustCore item reduces or removes the Anitta risk.

ANITTA TOPIC	TRUSTCORE FINDINGS	COMBINED EFFECT
A02 TrustCore token in app	R02, R03	Token holder can act beyond its role in TrustCore
A03 Legacy host	R10, R16	Legacy server also receives TrustCore's failure callbacks
A06 No advisor identity	R03, R08	TrustCore scoping cannot work without the caller's identity
A12 No status feedback	R17	Stuck offers in TrustCore are never surfaced
A13 Customer scoping	R03	Scoping depends on request parameters
Ordering via TrustCore	R01, R09	Unauthenticated webhooks can alter offers ordered from Anitta

D Glossary

Terms used throughout this manual.

TERM	MEANING
Agent service	Proposed backend that holds secrets, authenticates advisors and exposes TrustCore operations to the model as tools.
AI proxy	Neural Swiss gateway at ai.neuralswiss.ch that forwards Anitta's requests to the OpenAI Responses API.
Anonymous auth	Firebase sign-in without credentials; yields a uid per installation.
Dictation mode	Speech recognition mode optimised for continuous natural speech.
FlutterFlow	Visual builder that generates Flutter code; Anitta's shell is generated by it.
GetX	Flutter state-management and dependency-injection library used by TemplatesController.
Handle	TrustCore's identifier for a collection or template, e.g. template_nur_kvg.
PAD	Power Automate Desktop: the unattended robot that calculates offers on insurer portals.
Responses API	OpenAI API returning an output[] array of message, reasoning and tool items.
Sanctum token	TrustCore API bearer token (Laravel Sanctum personal access token).
Template	TrustCore configuration of product lines for an insurer; selected when ordering an offer.
Tool calling	Model capability to request execution of typed functions instead of only producing text.
TTS / STT	Text-to-speech / speech-to-text.